

Understanding the General-Purpose OS
Lifecycle of a Linux Process:
`'cp -vi .bashrc bashrc.bak'`

u5602780

April 22, 2025

Contents

1	Introduction	3
2	Understanding the General-Purpose OS	3
2.1	Definition and Role of General-Purpose OS	3
2.2	User Space vs Kernel Space	3
2.3	Introduction to Linux	4
2.4	Role of the Command Line	4
3	Lifecycle of the ‘cp -vi .bashrc bashrc.bak’ Process	4
3.1	Process Creation	4
3.2	Memory management	6
3.3	File Management, and Process Termination	7
3.4	User Interaction	9
3.4.1	Program flags	9
3.4.2	Input/Output Handling	9
4	Conclusion	10

1 Introduction

In this report I will explain the lifecycle of a Linux process within a general-purpose operating system, using the command `cp -vi .bashrc bashrc.bak` as a practical example. We will explore the fundamental concepts of the Linux operating system, including the distinction between user space and kernel space, the role of the command line, and the intricacies of process creation, memory management, file operations, and user interaction. This report will delve into the security controls inherent in the process lifecycle, examining file permissions, and the security boundaries enforced between user and kernel space.

2 Understanding the General-Purpose OS

2.1 Definition and Role of General-Purpose OS

An operating system (OS) is a layer of software that manages and controls a computer's hardware and software resources. It abstracts the complexity of hardware, providing users with a simplified interface for interaction.

Additionally, the OS offers a collection of libraries, and pre-written code modules, that enable applications to perform essential tasks without directly managing hardware.

The primary functions of an operating system include:

- **File Management** – Organising, storing, retrieving, and controlling access to files.
- **Process Management** – Handling process creation, execution, and termination, including multitasking and resource allocation.
- **Hardware Management** – Managing system resources such as CPU, memory, storage, and peripherals to ensure efficient operation.
- **User Interface Management** – Providing command-line or graphical interfaces for user interaction with the system.

2.2 User Space vs Kernel Space

User space can be described as a sandbox—a safe environment that allows different programs to run without directly impacting each other. This is achieved through **process isolation**, which involves separating the memory and execution spaces of different processes to ensure that one process cannot access or interfere with the memory of another. Additionally, **privilege separation** is dividing a program into distinct components, each operating with the minimum privileges necessary to perform its specific task. If a program in user space crashes, it is unlikely to affect other running programs.

Unlike user space, programs running in **kernel space** can directly interact with hardware and system memory. This means that when a fault occurs in the kernel, the entire system has a high chance of halting, leading to a *kernel panic*. During a kernel panic, the system dumps the current memory state into a log file for debugging purposes [Babar(2013)].

The Linux kernel is composed of modular components known as **kernel modules**. A method for hot-swapping kernel modules has been proposed by [Lee and Chang(2006)],

which allows faulty modules to be replaced without requiring a system restart, thereby preventing an entire system halt.

In conclusion, kernel mode is powerful and efficient but fragile, whereas user mode is safer due to enforced security measures but suffers from increased overhead and rigidity.

2.3 Introduction to Linux

Linux is an open-source, Unix-like operating system used in various environments, from personal computers and servers to embedded systems and IoT devices. It is known for its stability, security, and flexibility. Unlike proprietary OSs, Linux distributions (distros) such as Ubuntu, Debian, Fedora, and Arch Linux offer users different levels of control and customization. The Linux kernel, which manages hardware resources, is the foundation of the OS, with various shells and graphical interfaces built on top.

2.4 Role of the Command Line

The command line is an interface where users enter commands and data. It is a fundamental tool for interacting with Linux, offering precise control over system operations [GeeksforGeeks(2024b)]. Users can execute commands for file management (ls, cp, rm), process control (ps, kill), networking (ping, netstat), and system administration (sudo, apt, systemctl).

Modern Linux distributions include a shell and a command-line interpreter that acts as a bridge between the user and the system. The shell translates user input into system calls (syscalls) that interact directly with the kernel [GeeksforGeeks(2024a)]. Unlike users, the shell has access to the kernel via the system API, allowing it to manage system processes and resources efficiently.

Shell scripting, using Bash or other shells (C Shell, Korn Shell), enables the automation of repetitive tasks, making the command line indispensable for developers, system administrators, and cybersecurity professionals. Many Linux-based systems, especially servers, rely primarily on the command line for efficiency and remote management.

3 Lifecycle of the ‘cp -vi .bashrc bashrc.bak’ Process

3.1 Process Creation

When a user opens a shell, the shell operates under the user’s access rights, and any commands issued within that shell inherit these privileges. As soon as the command

cp -vi .bashrc bashrc.bak

If submitted, the Bash shell initiates its processing sequence. Before execution, multiple internal steps need to be performed.

The first step is to determine how many commands exist in a single line. The shell accomplishes this by splitting the input into tokens, where a token can be an entire command or a keyword such as **if**.

After tokenisation, the shell performs syntax checking on each token. Execution cannot proceed if there is a syntax error in any token.

Next, the shell determines whether the command is a built-in or an external command. If it is an external command, the shell searches for it in the directories specified by the **\$PATH** environment variable [GNU_Project(2022)].

To be able to understand when a child finishes execution, the program sets up a "signal handler" to be able to receive the SIGCHLD signal that shows that the child has finished its execution.

Once all the aforementioned steps are completed, the shell proceeds with execution. It creates a child process using the **fork()** system call. Within this child process, the shell invokes the **execve()** system call to replace the child process with the target program. After initiating execution, the shell waits for the child process to complete [Scribbles(2021)].

We can observe the system calls involved in the execution process by employing the **strace** utility, which traces system calls and signals [Kerrisk et al.(2025b)].

As example: The following output from **strace** demonstrates the usage of the **execve()** system call:

```
execve("/usr/bin/cp", ["cp", "-vi", ".bashrc", "bashrc.bak"], 0x7ffec9ffd588 /* 69 vars */) = 0
```

The return value of **0** following the system call in the **strace** output signifies successful execution. To understand the parameters and behaviour of this system call, we can consult the manual page using the **man** command. The **man** page provides a detailed description of the **execve()** system call, including its syntax:

```
int execve(const char *pathname, char *const _Nullable argv[],
           char *const _Nullable envp[]);
```

When the **execve()** system call is invoked, the following happens:

- The **.text**, **.data**, heap, and stack segments of the current process are discarded.
- All memory mappings, such as **mmap()** and **shmat()**, are removed.
- Kernel structures associated with the process (like the process ID, parent-child relationships, and file descriptors) remain intact, but the process's memory is wiped out.

Once the clean-up process is completed, the kernel loads the new executable into memory. New **.text**, **.data**, heap, and stack segments are created for the new process. Additionally, the environment (**envp**) and argument (**argv**) arrays are set up in the new process's memory.

In our case, **argv** will look like this:

```
argv = {
    "cp",           // argv[0] → The name of the program (by convention,
                    // usually the same as the binary name)
    "-vi",          // argv[1] → The '-vi' flag (verbose and interactive mode)
    ".bashrc",       // argv[2] → The source file to copy
    "bashrc.bak",    // argv[3] → The destination file
    NULL             // argv must end with a NULL pointer
};
```

And the **envp** will look like this:

```

envp = {
    "PATH=/usr/local/bin:/usr/bin:/bin",
    "HOME=/home/user",
    "SHELL=/bin/bash",
    "USER=username",
    ...
    (total 69 environment variables)
    NULL // envp must also end with a NULL pointer
};

```

[Kerrisk et al.(2018)]

3.2 Memory management

After the kernel has finished memory allocation, all subsequent memory management is handled by the program itself.

For example, **strace** command traces the memory allocation for shared libraries (e.g., `libselinux.so.1`) used by the program:

```

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=174312, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 186064, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fd8be6cb000
mmap(0x7fd8be6d2000, 110592, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7000) = 0x7fd8be6d2000
mmap(0x7fd8be6ed000, 32768, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x22000) = 0x7fd8be6ed000
mmap(0x7fd8be6f5000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x29000) = 0x7fd8be6f5000
mmap(0x7fd8be6f7000, 5840, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fd8be6f7000
close(3) = 0

```

The **mmap()** system call is used to map the library into the process's virtual address space, ensuring that the code and data are accessible [Michael Kerrisk and others(2024)]. The kernel allocates memory on demand, using **demand paging** to load only the required pages into physical RAM, reducing memory waste.

After multiple libraries are loaded, we see memory protection changes:

```

mprotect(0x7fd8be6a5000, 16384, PROT_READ) = 0
mprotect(0x7fd8be4d5000, 4096, PROT_READ) = 0
mprotect(0x7fd8be6be000, 4096, PROT_READ) = 0
mprotect(0x7fd8be6c9000, 4096, PROT_READ) = 0
mprotect(0x7fd8be6f5000, 4096, PROT_READ) = 0
mprotect(0x55e7ddc34000, 4096, PROT_READ) = 0

```

To safeguard the `cp` process, Linux employs several protection mechanisms. The **mprotect()** system call sets permissions on memory regions, ensuring that the text segment is read-only (`PROT_READ`) to prevent code modification, while the data and stack segments allow read/write access. For instance, the `mmap()` call above specifies **PROT_READ|PROT_WRITE** for the library's data segment. Additionally, **Address Space Layout Randomization** (ASLR) randomises the base addresses of the text, data, heap, and stack segments, making it harder for attackers to exploit memory vulnerabilities. The kernel also enforces page permissions, preventing the `cp` process from accessing memory outside its allocated regions, thus ensuring process isolation.

These mechanisms ensure that the `cp` process efficiently manages memory while maintaining security and stability during file operations.

If there is not enough memory for the process to finish or to load `/usr/bin/cp` executable, one of the two errors will be raised:

- **ENOMEM**: Not enough memory to complete the operation.
- **ENOSPC**: No space left on the device.

Before returning **ENOSPC**, the kernel attempts to move pages into swap memory. Swap plays the role of "slow RAM" when the system uses disk space as RAM. This is possible due to the usage of pages:

- Every process gets a virtual address space that may exceed available physical RAM.
- The Memory Management Unit (MMU) translates virtual addresses to physical addresses.
- The kernel allocates pages on demand and uses demand paging to load only necessary parts of a process into memory.

If a process needs more memory but the RAM is full, the kernel may:

- Reclaim unused memory.
- Swap inactive pages to disk.
- Kill processes (OOM killer) if memory is critically low.

[Kumar(2023)]

3.3 File Management, and Process Termination

To transfer data, the program opens the `.bashrc` file in read-only mode which is set by the parameter **O_RDONLY**.

```
openat(AT_FDCWD, ".bashrc", O_RDONLY) = 3
```

To retrieve the file's data, such as permissions and size we use **newfstatat** system call.

```
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=803, ...}, AT_EMPTY_PATH) = 0
```

Here is the breakdown for returned data:

- **st_mode=S_IFREG|0644**: This indicates that the file is a regular file (**S_IFREG**) with permissions **0644** (**rw-r--r--**).
- **st_size=803**: The file size is 803 bytes.
- **AT_EMPTY_PATH**: This flag allows querying the file linked to **dirfd=3** even if the path is empty (`""`). It means "use the file descriptor directly."

Next, the program calls the **open()** syscall with the parameters **O_WRONLY|O_CREAT|O_EXCL**:

- **O_WRONLY**: Open the file for write-only access.

- **O_CREAT**: If the file doesn't exist, create it.
- **O_EXCL**: If the file already exists, fail the operation (don't overwrite the file). This ensures that a new file is created, and if **bashrc.bak** already exists, the open operation fails.

The file **bashrc.bak** is created with the permissions of read and write for root, and read for everyone else:

```
openat(AT_FDCWD, "bashrc.bak", O_WRONLY|O_CREAT|O_EXCL, 0644) = 4
```

The return value of **4** is the file descriptor for **bashrc.bak**.

It is important to note that the **0644** parameter in the **openat()** syscall is not the only factor determining read-write access to the file. The **umask** (user file-creation mask) defines which permission bits should be removed from newly created files. [Kerrisk et al.(2025c)] On most Linux systems, the **umask** value is **0022**, which is applied to every file created via **open()** or **mkdir()**.

Therefore, as an example if we call **open()** syscall with 0666 parameter in access control field (i.e., everyone can read and write) if the **open()** call had looked like this:

```
openat(AT_FDCWD, "test.txt", O_WRONLY|O_CREAT|O_EXCL, 0666)
```

the permissions on the new file would be **0644** (**0666 XOR 0022 = 0644**). This mechanism is implemented as a security measure to prevent newly created files from being inadvertently granted overly permissive access.

After the **bashrc.bak** file is created, the **ioctl** syscall is called, which efficiently makes a copy of the file with descriptor 3 (**.bashrc**) into a file with descriptor 4 (**bashrc.bak**) [Kerrisk et al.(2025a)]:

```
ioctl(4, BTRFS_IOC_CLONE or FICLONE, 3) = 0
```

After copying the files, the program closes the file descriptors:

```
close(4) = 0
close(3) = 0
```

The last steps of the program are to close all files and call the **exit** command:

```
close(0) = 0
close(1) = 0
close(2) = 0
exit_group(0) = ?
```

The **exit_group()** syscall terminates all threads of the process. It clears the **.data**, **.text** sections, and heap and stack, releasing all memory. The parent process is notified by receiving a **SIGCHLD** signal. Since the parent (shell) was waiting (**waitpid()**), it receives the exit status.

After receiving this signal, the shell takes over the user's input and starts waiting for new inputs.

3.4 User Interaction

3.4.1 Program flags

the flags in the command

```
cp -vi .bashrc bashrc.bak
```

are handled by the **cp** program are **v** and **i**. They mean:

-v (verbose): Displays the files being copied. In our case, the output looks like:

```
' .bashrc' -> 'bashrc.bak'
```

-i (interactive): Prompts before overwriting an existing file. If **bashrc.bak** already exists, the prompt will appear as:

```
cp: overwrite 'bashrc.bak'?
```

The user must respond with **y** or **yes** to continue. Any other input will interrupt the execution and the file will not be overwritten.

3.4.2 Input/Output Handling

In Linux, input and output (I/O) are managed under the principle that *everything is a file* — including hardware devices (e.g., keyboards, disks) and virtual interfaces (e.g., pipes, sockets). Each running process is by default assigned three standard file descriptors: **stdin** (standard input, file descriptor 0), **stdout** (standard output, fd 1), and **stderr** (standard error, fd 2). These descriptors are inherited from the parent process — typically the shell — and initially connect to the user’s terminal [Cane(2012)].

Programs interact with these descriptors via system calls such as **read()**, **write()**, **open()**, and **close()**. For instance, a command like **echo "hello"** writes the string to **stdout**, which the terminal then displays. Programs requesting user interaction (e.g., **cp** with the **-i** flag) read from **stdin**. Separating **stderr** from **stdout** allows programs to manage error reporting independently from the regular output, enabling clean logging and redirection.

The shell provides a rich syntax for I/O redirection: **>** redirects **stdout** to a file, **<** reads **stdin** from a file, and **2>** redirects **stderr**. For example, **command > output.txt 2> error.txt** separates standard output and error messages into distinct files. Piping (**|**) allows the **stdout** of one command to be used as the **stdin** for another, e.g., **ls | grep txt** [McConnell(2017)].

At a lower level, Linux manages hardware interaction through device files in **/dev** (e.g., **/dev/null**, **/dev/tty**, **/dev/sda**). These are special files that interface with kernel device drivers. User-space programs never access hardware directly; instead, the kernel abstracts these operations to maintain consistency, security, and portability [Baranwal(2023)].

Processes created by the shell inherit its I/O context. While interactive programs often use these standard descriptors, background services (daemons) and graphical applications may close them or redirect output to log files or system logging services.

4 Conclusion

Executing a program in Linux, exemplified by `cp -vi .bashrc bashrc.bak`, involves a complex sequence of steps, initiated by `fork()` and `execve()`. They are managed through system calls like `mmap()`, `mprotect()`, `openat()`, and `ioctl()`. Throughout memory management, file operations, and I/O handling, numerous security mechanisms—including process isolation, privilege separation, and file permission enforcement via `umask`—are crucial for maintaining system integrity and preventing unauthorised access. The kernel’s role in mediating hardware interaction and enforcing security boundaries is paramount. Understanding these intricate processes and security controls is essential for robust and secure application development within the Linux environment.

References

- [Babar(2013)] Yogesh Babar. *Understanding How Kdump Works*, April 2013. URL https://access.redhat.com/sites/default/files/attachments/kdump20130402_1.pdf. Technical Reviewer: Chris Negus.
- [Baranwal(2023)] Vinayak Baranwal. Linux dev directory explained: Manage device files effectively, 2023. URL <https://www.voxfor.com/linux-dev-directory-explained-manage-device-files-effectively/>. Accessed: 2025-04-04.
- [Cane(2012)] Benjamin Cane. Unix shell: The art of i/o redirection, 2012. URL <https://web.archive.org/web/20220629044814/http://bencane.com:80/2012/04/16/unix-shell-the-art-of-io-redirection/>. Accessed: 2025-04-04.
- [GeeksforGeeks(2024a)] GeeksforGeeks. Introduction to linux shell and shell scripting, 2024a. URL <https://www.geeksforgeeks.org/introduction-linux-shell-shell-scripting/#what-is-kernel>. Last updated: 16 Apr, 2024.
- [GeeksforGeeks(2024b)] GeeksforGeeks. Difference between terminal, console, shell, and command line, 2024b. URL <https://www.geeksforgeeks.org/difference-between-terminal-console-shell-and-command-line/>. Last updated: 23 Sep, 2024.
- [GNU_Project(2022)] GNU_Project. *The GNU Bash Reference Manual*, 5.2 edition, September 2022. URL <https://www.gnu.org/software/bash/manual/bash.html>. Accessed: 2025-03-24.
- [Kerrisk et al.(2018)] Michael Kerrisk et al. `execve(2)` - execute a file. <https://manpages.ubuntu.com/manpages/bionic/en/man2/execve.2.html>, 2018. Provided by: manpages-dev_4.15-1.all, Accessed: 2025-03-26.
- [Kerrisk et al.(2025a)] Michael Kerrisk et al. `ioctl(2)` - *Linux manual page*, 2025a. URL <https://www.man7.org/linux/man-pages/man2/ioctl.2.html>. Accessed: 2025-03-24.

- [Kerrisk et al.(2025b)] Michael Kerrisk et al. *strace(1) - Linux manual page*, 2025b. URL <https://www.man7.org/linux/man-pages/man1/strace.1.html>. Accessed: 2025-03-24.
- [Kerrisk et al.(2025c)] Michael Kerrisk et al. *umask(2) - Linux manual page*, 2025c. URL <https://www.man7.org/linux/man-pages/man2/umask.2.html>. Accessed: 2025-03-24.
- [Kumar(2023)] Satish Kumar. Process memory management in linux, 2023. URL <https://www.tutorialspoint.com/process-memory-management-in-linux>. Updated on: 24-Mar-2023, Accessed: 2025-03-24.
- [Lee and Chang(2006)] Yueh-Feng Lee and Ruei-Chuan Chang. Hotswapping linux kernel modules. *Journal of Systems and Software*, 79(2):163–175, 2006. ISSN 0164-1212. doi: <https://doi.org/10.1016/j.jss.2005.05.011>. URL <https://www.sciencedirect.com/science/article/pii/S0164121205001068>.
- [McConnell(2017)] Roslyn McConnell. Pipes, forks, & dups: Understanding command execution and input/output data flow, 2017. URL <https://www.rozmichelle.com/pipes-forks-dups/>. Accessed: 2025-04-04.
- [Michael Kerrisk and others(2024)] Michael Kerrisk and others. mmap(2) — linux manual page. <https://www.man7.org/linux/man-pages/man2/mmap.2.html>, 2024. Accessed: 2025-04-22.
- [Scribbles(2021)] Beta Scribbles. An overview of the working of the bash shell. *Geek Culture*, August 2021. URL <https://medium.com/geekculture/an-overview-of-the-working-of-the-bash-shell-f063e7f09945>. Accessed: 2025-03-24.